

Create Block Storage In Oracle OCI

Step 1: Navigate to the Block Volume Section

- Go to **Storage** → **Block Volumes** → **Create Block Volume**

Setp 2: Create a New Block Volume

- Provide a **name** for the block volume.
- Set the **Compartment** to **ptpl (root)**.
- Select the **Availability Domain** (based on the tenancy region).
- Configure the **Volume Size and Performance**:
 - Choose **Custom Configuration** and specify the required volume size.
 - Enable **Target Volume Performance**.
 - Set up **Default VCPUs/GB** and **Maximum VPUs/GB**.
 - Turn on **Detached Volume Auto-Tune**.
- Keep all other settings as default and **Create Block Volume**.

Setp 3: Attach the Block Volume to an Instance

- After creation, navigate to **Resources** → **Attached Instances**.
- Click **Attach to Instance**.
- Select the running instance where you want to attach the volume.

Mount the Attached block volume to server and file system

Step 1: Go to the instance where the created volume attached

Step 2: Go to root and run the command **lsblk** , you will able to see the mounted BV below with Name (example: sda / sdb)

Step 3: Run the below commands to create the new partition on block storage and mount that to file system

- `sudo fdisk /dev/sdb`
- `Sudo mkfs.ext4 /dev/sdb1`
- (The above two commands need to be run only when the BV setup done in new instance, else be careful the above command may erase the data's present in it)
- `Sudo mkdir -p /data/Prod`
- `Sudo mount /dev/sdb1 /data/Prod`
- `df -h` (This should show the mounted BV in the file system)

Step 3: To ensure the attached BV must not unmount after a reboot, follow the below steps

- Recive the UUID of the created BV: **run the command blkid** , you will get the see the UUID below

```
private:/# blkid
1: LABEL="cloudimg-rootfs" UUID="b4d45fd3-9ab8-4ddb-934c-184084c249d0" TYPE="ext4" PARTUUID="6fe4381e-aea7-44e3-a830-abcde608f41a"
15: LABEL="FATBOOT" LABEL="UEFI" UUID="7CEA-00EC" TYPE="vfat" PARTUUID="779ead57-c86c-4df0-95c3-9199cc62f347"
0: TYPE="squashfs"
01: TYPE="squashfs"
02: TYPE="squashfs"
03: TYPE="squashfs"
04: TYPE="squashfs"
05: TYPE="squashfs"
06: TYPE="squashfs"
07: TYPE="squashfs"
08: TYPE="squashfs"
09: TYPE="squashfs"
1: UUID="27682bed-0480-4127-bf1a-47ecfef54020" TYPE="ext4" PARTUUID="44d3d6ca-01"
010: TYPE="squashfs"
1: UUID="106318bb-b0a1-4626-a53e-69c65824179a" TYPE="ext4" PARTUUID="90e7bbe5-01"
14: PARTUUID="4c5b942a-8a60-4664-99d5-958124f4b1b7"
```

Step 4: Enter the UUID under `/etc/fstab` with the partition name which is created as shown in below snapshot

```
#KRGPROD
UUID=27682bed-0480-4127-bf1a-47ecfef54020 /data/ProdKRG ext4 defaults 0 2
```

Step 5: Reload the mount

- `Sudo mount -a`

Mount the created BV as NFS server and shared it in other application server

Step 1: Create the new ubuntu VM for NFS server and install the below mentioned services in it

- nfs-server
- nfs-common
- nfs-kernel-server
- rpcbind
- rpcbind.socket

And allow the below mentioned port in OCI level and VM level

- 111 /tcp
- 111 /udp
- 2049 /tcp

Step 3: Check the status of all the service

Step 4: Attached the BV to the NFS server and follow the same steps provided in earlier method

Step 5: Once after configuring the above method with BV mount to file system, export the created partition under **/etc/exports** as shown below

```
#KRGPROD  
  
/data/ProdKRG 10.0.1.0/24(rw,sync,no_root_squash,no_subtree_check)
```

- /data/ProdKRG is the created partition
- 10.0.1.0/24 is the subnet CIDR block of the VM
- Rw,sync,no_root_squash,no_subtree_check is the exports flags

Step 6: Reload the exports

- **sudo exportfs -v**

Step 7: Now mount the drive from the client server with below commands

- Create the partition in client server with same name as created in NFS server
- Run the below command to mount in client server
 - **sudo mount -t nfs <private-ip of NFS server>:/data/Prod /data/Prod**

Step 8: Check the mounted successful with **df -h** command

```
10.0.1.73:/data/ProdKRG 492G 384G 83G 83% /data/ProdKRG
```